HR GENIUS

Human Resource Management System

Project Synopsis, Technical Details & API Documentation

Project type	Full Stack Development
Industry domain	Human Resources
Backend	Java 21 · Spring Boot 3 · Spring Data JPA / Hibernate
Frontend	Angular · TypeScript
Database	Oracle Database · SQL · PL/SQL
Security	Spring Security · JWT · BCrypt · Role-based access
Dataset	17 synthetic CSV files, separate from source code

Project repository: https://github.com/arao1118/HR-Genius

1. Project Synopsis

HR Genius is a Human Resource Management System (HRMS) that supports the employee lifecycle: employee data, recruitment and onboarding, payroll, performance management and HR analytics. The supplied project catalogue specifies Java, Spring Boot/Hibernate, Angular and Oracle Database, within the Human Resources domain.

The database design follows a record-keeping core: Employees are the central records, with connected modules for employment history, education, recruitment, interviews, attendance, leave, salary, payroll, performance reviews and access control.

2. Problem Statement

Employee information and HR workflows can become fragmented across separate records and manual processes. HR Genius provides a shared application model for employee master data and HR transactions, with relational constraints, approval workflows, role-based access and auditability.

3. Objectives

- Maintain employee demographic details, education qualifications and employment history.
- Manage job requisitions, candidates, interview scheduling, offers and onboarding workflow.
- Record attendance and manage leave types, requests, approvals and balances.
- Maintain salary components and employee salary structures; process period-based payroll and payslip summaries.
- Support performance review cycles, reviewer ratings, feedback and goals.
- Provide workforce reports for headcount, attrition, leave and payroll cost.
- Protect HR data through authentication, role permissions, password hashing, restricted data access and audit logs.

4. Functional Scope

Module	Description / key features
Employee Registration & Demographics	Employee profile, demographics, education and employment history, document upload and emergency contacts.
Recruitment & Onboarding	Job requisitions, candidate tracking, interview scheduling, offer letters and onboarding checklist.
Attendance & Leave	Attendance logs, leave policies, leave requests and approvals, holiday calendar.
Payroll	Salary structure, earnings and deductions, tax/PF policy, monthly payroll run and payslip generation.
Performance Management	Goal setting, review cycles, self/manager ratings, feedback notes and review forms.
HR Analytics & Reporting	Headcount, attrition, leave and payroll-cost reports, dashboards and exports.
User Management & Access Control	User creation, role assignment, deactivation, login audit and password policy.

5. Technology Stack and Architecture

Backend: Java 21, Spring Boot 3, Spring Web, Spring Data JPA, Hibernate, Bean Validation and Spring Security. The architecture uses REST controllers, service-layer business logic, repositories and JPA entities.

Frontend: Angular standalone application with TypeScript, a responsive dashboard shell and an HTTP service for backend communication.

Database: Oracle relational database. The supplied package includes Oracle table DDL and representative PL/SQL procedures.

Security: JWT bearer authentication, BCrypt password hashing and role-based authorization. Employee-self and manager-team data scoping must be enforced throughout the service/query layer before production.

6. Database Design

The supplied design identifies seventeen core tables (Roles and Users counted separately). The implementation extends the schema with AuditLogs.

- Employees, Departments, Designations, EmploymentHistory, EducationQualifications.

- JobRequisitions, Candidates, Interviews.
- AttendanceRecords, LeaveTypes, LeaveRequests.
- SalaryComponents, EmployeeSalary, Payroll.
- PerformanceReviews, Roles, Users.
- AuditLogs — implementation extension.

The source document specifies unique EmployeeCode and Email, a leave date check (ToDate must be on or after FromDate), required keys and dates, and indexes for foreign keys and frequently filtered fields such as EmployeeID, PayPeriod and Status.

7. Business Rules and Workflows

- Employee registration validates duplicate employee code and email; referenced department/designation/manager records must exist.
- Job requisitions are created in Pending Approval status and should be approved before posting.
- Interview scheduling should check interviewer availability and prevent conflicts.
- Leave requests must have valid date ranges; overlapping approved leave is rejected by the current service implementation. Managers approve or reject with comments.
- Payroll is period-based. The current demonstration service skips an employee-period already processed and marks new payroll rows LOCKED.
- Performance ratings are stored per reviewer; the specification says results become visible after HR closes the review cycle.
- Deactivated users cannot log in; access is role-controlled.

8. Views and Stored Procedures

The supplied design proposes EmployeeSummary, ManagerTeamSchedule, PayrollHistory, RecruitmentPipeline, LeaveBalance and PerformanceOverview views. It also lists RegisterEmployee, CreateJobRequisition, ScheduleInterview, ConvertCandidateToEmployee, ApplyLeave, ApproveLeave, RunPayroll, CreatePerformanceReview, CloseReviewCycle and UserManagement procedures.

The delivered Oracle SQL includes schema DDL and representative PL/SQL procedures for employee registration, job requisition creation and leave application. The remaining procedures are documented as future implementation work.

9. Security, Privacy and Compliance

- Store salted password hashes only; do not store plaintext passwords.
- Restrict salary, bank and identity information; mask sensitive values in reports and encrypt sensitive data at rest and in transit.
- Employees should see only their own data; managers should see their team; HR and Payroll Admin access should be scoped.
- Audit create/update/delete operations on employee, payroll, leave and user records with actor and timestamp.
- Apply tax and provident-fund rules according to the organisation’s policies and applicable law.
- Use constraints, transactions, indexes, backups and retention policies to preserve integrity.

10. API Documentation

Base URL (local): **http://localhost:8080/api**. Requests and responses use JSON unless otherwise noted. Protected endpoints require an Authorization header with a bearer JWT: **Authorization: Bearer <accessToken>**.

Authentication — POST /auth/login: authenticate an active user and issue a JWT.

```
{
"username": "hr.admin",
"password": "your-password"
}
```

Success (200):

```
{
"accessToken": "&lt;signed-jwt&gt;",
```

```
"tokenType": "Bearer"
}
```

Health — GET /health: public health check; no authentication required.

```
{
"status": "UP",
"application": "HR Genius"
}
```

Employee APIs: GET /employees lists employees (HR_ADMIN, HR_EXECUTIVE, MANAGER); GET /employees/{id} retrieves one employee (same roles); POST /employees creates an employee (authenticated); PUT /employees/{id} updates supported profile fields (authenticated). Duplicate employee code/email are rejected.

POST /employees example:

```
{
"employeeCode": "HG-2001",
"firstName": "Anaya",
"lastName": "Mehta",
"email": "anaya.mehta@example.test",
"hireDate": "2026-10-31",
"status": "ACTIVE",
"department": { "id": 1 },
"designation": { "id": 1 }
}
```

Leave APIs: GET /leaves lists leave requests; POST /leaves creates a request; PATCH /leaves/{id}/decision approves/rejects a pending request. These currently require authentication.

POST /leaves example:

```
{
"employeeId": 3,
"leaveTypeId": 1,
"fromDate": "2026-11-10",
"toDate": "2026-11-12",
"reason": "Personal time"
}
```

PATCH /leaves/{id}/decision example:

```
{
"approverId": 2,
"approve": true,
"comment": "Approved"
}
```

The service rejects invalid date ranges, overlap with approved leave and decisions against non-pending requests. Leave balance/quota enforcement and manager-team authorization are not yet implemented.

Payroll APIs: GET /payroll lists payroll rows; POST /payroll/run processes employees not yet processed for a period.

POST /payroll/run example:

```
{
"payPeriod": "2026-10"
}
```

The period must match YYYY-MM. The demonstration service sums configured EARNING components and DEDUCTION components, calculates net pay as gross minus deductions, skips existing employee-period entries and marks new rows LOCKED. It does not implement jurisdictional statutory calculations, effective-date filtering, proration, attendance adjustments or statutory rounding.

Error responses: IllegalArgumentException maps to HTTP 400 and IllegalStateException to HTTP 409. Example:

```
{
"timestamp": "2026-10-31T08:00:00Z",
"error": "Employee code already exists"
}
```

Not yet exposed as REST APIs: recruitment, interview scheduling, attendance, performance, salary setup, education/employment history, role administration, audit logs, reporting views and CSV exports. The corresponding models/repositories/schema elements are present in the starter, but controllers/workflows are future work. Do not treat them as available routes.

11. Dataset Files and Expected Project Outputs

The separate dataset package contains 17 UTF-8 CSV files with synthetic records. They model HR Genius entities and can be used for local import, API testing and demo screens. They are sample seed/reference data, not actual production outputs generated by a running deployment.

CSV file	Contents / use
departments.csv	Department lookup records
designations.csv	Designation and grade lookup records
employees.csv	Synthetic employee master records
employment_history.csv	Prior employment per employee
education_qualifications.csv	Education records
job_requisitions.csv	Open and pending job requisitions
candidates.csv	Candidate pipeline stages
interviews.csv	Interview schedule/result samples
attendance_records.csv	Daily attendance entries
leave_types.csv	Leave policy lookup
leave_requests.csv	Leave requests and decisions
salary_components.csv	Earning/deduction definitions
employee_salary.csv	Employee salary component amounts
payroll.csv	Sample payroll summaries
performance_reviews.csv	Review-cycle ratings and feedback
roles.csv	RBAC roles
users.csv	Demo user mapping with placeholder password hashes

All people, contact details and example.test email addresses are fabricated. users.csv uses REPLACE_WITH_BCRYPT_HASH placeholders and is not a valid login seed. Do not use this data for real HR decisions or payroll.

12. Setup and Execution

- Install JDK 21 and Maven 3.9+; configure Oracle Database and create a schema user.
- Set HR_DB_URL, HR_DB_USERNAME, HR_DB_PASSWORD and HR_JWT_SECRET. The default dev profile uses in-memory H2 for smoke tests.
- Run backend using mvn spring-boot:run. Local API base: http://localhost:8080/api.
- Use backend/src/main/resources/db/oracle-schema.sql to create the Oracle schema. Oracle profile uses ddl-auto=validate.
- Install Node.js 20+; from frontend/ run npm install and npm start. The dashboard expects the backend at localhost:8080.

13. Limitations and Future Enhancements

- Complete REST APIs and Angular workflows for every module; add search, pagination, filtering and export.
- Implement manager-team and employee-self authorization; add CORS allow-list and token lifecycle controls.
- Add Flyway/Liquibase migrations, comprehensive unit/integration/API tests and Oracle compatibility verification.
- Implement approved payroll policy with tax/PF/ESI calculations, effective dates, proration and payslip generation.
- Implement leave quota/accrual/carry-forward, holidays and concurrency-safe overlap validation.
- Add audit interceptors, secure document storage, notifications, observability, backups and disaster recovery.

14. Conclusion

HR Genius maps the supplied HRMS functional scope and database design into a Java/Spring Boot, Angular and Oracle project foundation. The source package, API reference and synthetic datasets provide a starting point for implementation, demonstration

and further testing. Production use requires completion of the listed workflows, security controls, statutory configuration and validation.

15. GitHub Repository

The project source repository is available at the following address:

https://github.com/arao1118/HR-Genius

Use the repository to review the project files, implementation and future updates.